# The 3 Traps of Naive Semantic Caching

- **Flat Vector Namespaces**

  Embedding raw prompts without tenant partitioning leaks private enterprise data.

- **Flat 0.88 Cosine Threshold**

  Conversations need loose match (0.85); exact factual queries require 0.96+.

- **The Stale Invalidation Race**

  ANN graphs cannot prune instantly when source docs mutate in PostgreSQL.

- **Unchecked Stochastic Fallbacks**

  Failed upstream completions get cached and continuously served to clients.



By Gautam Sethi · AI Enthusiast

# The Partitioned Compound Key Topology

- **Compound Tenant Hash**

  Key: SHA256(Tenant_ID + Role_Clearance + Routing_Tier) + Prompt_Vector.

- **Sub-2ms Exact Hash Tier**

  Redis cluster catches identical high-volume queries with zero vector lookups.

- **Isolated Vector Partitions**

  Vector search runs strictly within the cryptographic tenant namespace.

- **Dynamic Similarity Gates**

  Tight 0.96+ threshold for factual queries, 0.86 for conversational queries.

# Solving the 500ms Invalidation Race

- **Change Data Capture (CDC)**

  Postgres Debezium pipeline triggers Kafka events when source docs mutate.

- **Document Fingerprint Vectors**

  Cached entries store hash fingerprints of all source documents used.

- **Optimistic Gateway Read Check**

  Gateway verifies doc hashes in Redis before returning any cached response.

- **Sub-50ms Global Eviction**

  Stale cache entries are purged before in-flight queries complete.

# Production Readiness Checklist

- **Physical Tenant Namespace**
  Vector collections physically partitioned by tenant, not soft filters.
- **Role-Based Access Enforcement**
  Cached answers inherit the minimum clearance level of their source chunks.
- **Negative Caching Circuit Breaker**
  Errors, timeouts, and fallback responses are strictly barred from cache.
- **PII & Dynamic Scrubbing**
  Session tokens, user IDs, and timestamps are stripped before embedding.